The Grader — LLM-as-Judge Evaluation Harness for Customer-Support Replies

Project report · Repository: github.com/TahaNw/Taha_grader · Stack: Python, Ollama, Qwen2.5 7B, pydantic, scikit-learn

1. Overview

The Grader is an evaluation harness that generates customer-support replies from a customer message and a company policy using a local Qwen2.5 7B model, then scores each reply from 1 to 5 with an LLM judge (correctness, completeness, politeness, clarity). It then evaluates the judge itself against a 150-item human-labelled golden dataset, with agreement, position-bias, verbosity-bias, latency and cost experiments.

2. Pipeline: how the whole project works

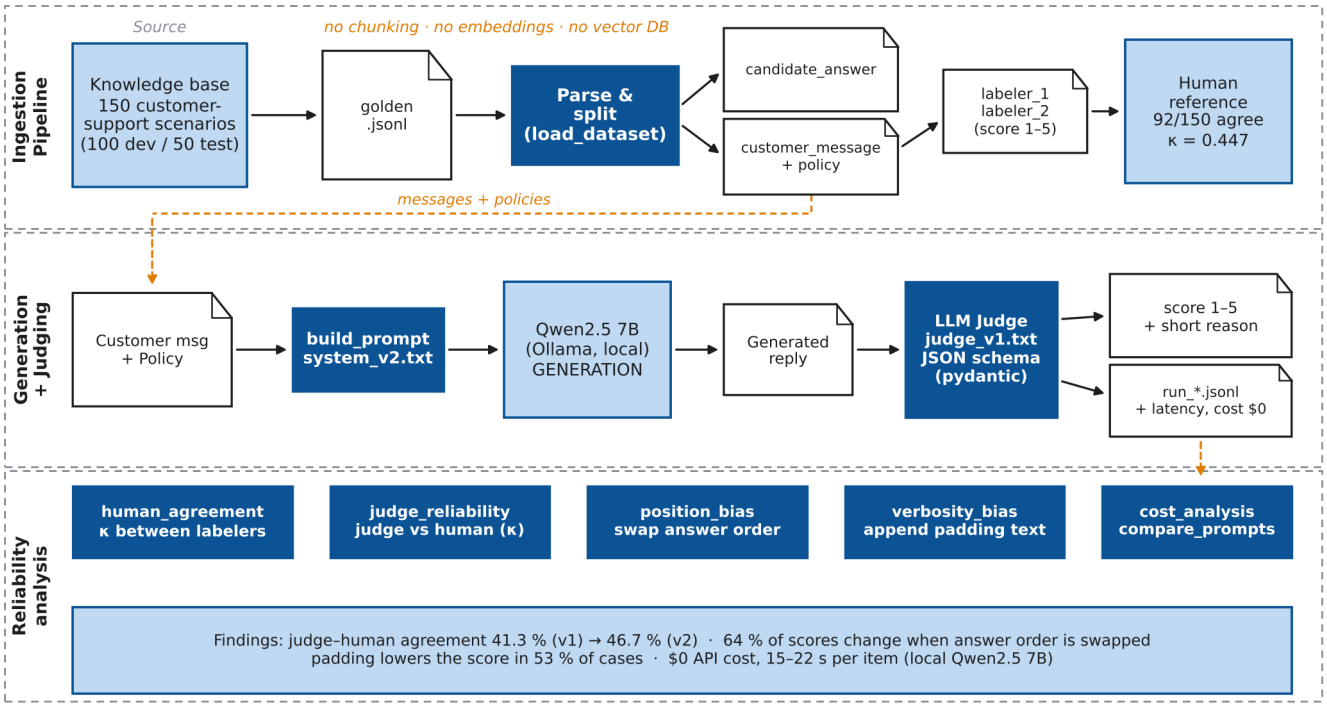


Figure 1 — End-to-end pipeline, drawn in the same lane style as the reference RAG diagram. Lane 1 loads and labels the data, lane 2 generates and judges replies, lane 3 audits the judge.

3. Embedding and chunking technique

None is used, and I am stating this rather than inventing one. This repository is an evaluation harness, not a RAG system: there is no embedding model, chunking strategy, TF-IDF/BM25 index, reranker or vector store anywhere in the code (checked by searching the whole repo). The equivalent of "ingestion" is loading *data/golden.jsonl* line by line; each item is already atomic (one short message and one short policy), so nothing needs splitting. The policy is injected whole into the prompt with *str.format* (context stuffing), which works because policies are one or two sentences.

4. Generative part

- **Generator:** Qwen2.5 7B served locally by Ollama (*ollama.chat*), no API key. Prompt *system_v1* (baseline) vs *system_v2* (answer every part, no invented prices, deadlines or guarantees, final policy-support self-check, no repetition).
- **Judge:** the same Qwen2.5 7B with *judge_v1.txt*; output is forced into a JSON schema (*score* 1–5 + *reason*) and validated with pydantic. Every result is appended to *results/run_*.jsonl* with generation, judge and total latency.
- **Dataset:** 150 scenarios (100 dev, 50 test), two independent human labels each, covering refunds, shipping, payments, subscriptions, privacy and insufficient-information cases.

5. Comparison table (this project has no vector DB, so the comparison is prompt v1 vs v2)

Measure	Run with system v1	Run with system v2	Change
Mean test score (50 items)	4.480	4.600	+0.120
Score 4 / Score 5 count	26 / 24	20 / 30	−6 / +6
Judge-human exact agreement (92 items)	41.30 %	46.74 %	+5.44 pp
Cohen's κ, judge vs human	0.178	0.202	+0.024
Generation latency / item	7.04 s	9.47 s	+2.43 s
Judge latency / item	8.30 s	12.05 s	+3.75 s
Total harness latency / item	15.34 s	21.52 s	+6.18 s
Direct API cost	$0	$0	—

If policies ever outgrew the prompt, a retrieval layer (embeddings plus a vector store such as Qdrant in the reference diagram) would become necessary; the current design does not need it.

6. Figures

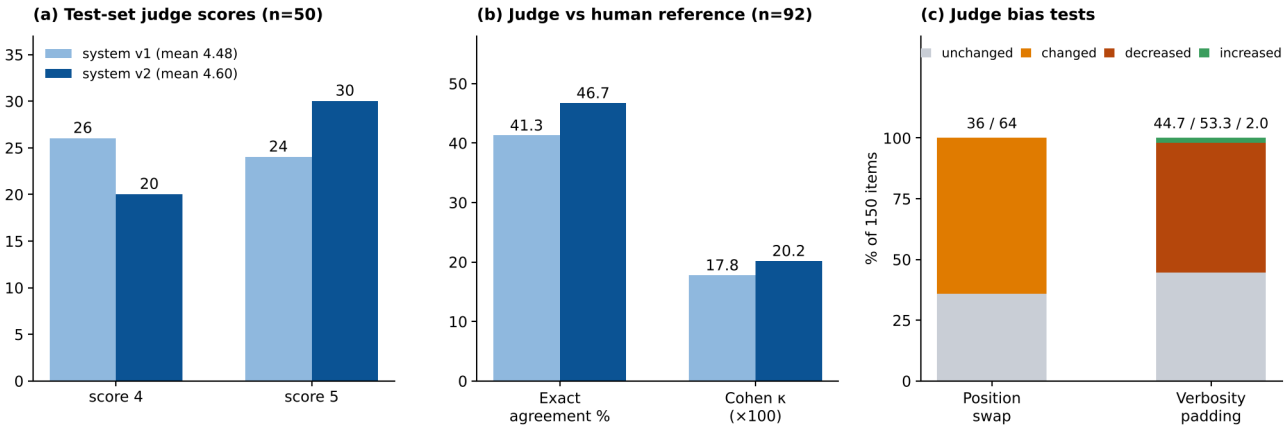


Figure 2 — (a) the v2 prompt shifts six test items from 4 to 5; (b) judge/human agreement improves modestly; (c) 64 % of scores change when answer order is swapped, and irrelevant padding lowers the score in 53 % of cases (it raises it in only 2 %).

7. What I learned

- **The evaluator must be evaluated.** Humans agreed on only 92/150 items (61.3 %, κ = 0.447), and the judge matched that agreed set just 41–47 % of the time (κ ≈ 0.2). One quality score hides this.
- **LLM judges are biased.** Order swaps moved 64 % of scores (mean change 0.48 points), so a single-pass judgement is not stable. The judge did not reward padding (2 % up, 53 % down), which is a good sign.
- **Quality costs time.** The stricter v2 prompt gained +0.12 mean score but added about 6 s per item; at 15,000 requests v2 generation alone is about 39 h. Local inference means $0 API cost but heavy compute time.
- **Limitations I noticed in the code:** the same Qwen2.5 7B both writes and judges the replies (self-preference risk); the human labels score the dataset's *candidate_answer*, whereas judge reliability compares them with the judge's score on Qwen's own generated reply, so the two are not rating identical text; and the test set is small (50 items, scores only 4 or 5), so +0.12 is suggestive rather than conclusive.

8. How to run it

- *ollama pull qwen2.5:7b*, then *pip install -r requirements.txt* (ollama, pydantic, pandas, scikit-learn, python-dotenv).
- *py -m harness.runner* generates and judges all 150 items and writes *results/run_<timestamp>.jsonl*; then run any script in *analysis/* (e.g. *py analysis/judge_reliability.py*).